Code Explanation

SQL Generator for Sales Orders

This Python script generates SQL queries for creating intermediate staging and main sales orders tables based on the provided mapping document. The generated SQL queries are then executed using Apache Spark.

Overview
The script consists of four main functions: `generate_sales_orders_stg_sql`, `generate_sales_orders_sql`, `execute_sql`, and `main`. The `generate_sales_orders_stg_sql` and `generate_sales_orders_sql` functions generate SQL queries for creating the intermediate staging table and the main sales orders table, respectively. The `execute_sql` function executes a given SQL query using Apache Spark. The `main` function orchestrates the entire process by generating and executing the SQL queries.

Step 1: Generating SQL for Intermediate Staging Table
The `generate_sales_orders_stg_sql` function generates a SQL query for creating the `sales_orders_comp_stg` intermediate table. This table is created by joining two tables, `ord_hdr` and `ord_dtl`, from the `b_source` database.

```
sql = """
CREATE OR REPLACE TABLE sales_orders_comp_stg AS
SELECT
    CONCAT(a.referenceto, a.refno) AS CompOrderNumber,
    ...
FROM
    (SELECT * FROM b_source.ord_hdr WHERE delete_flag <> 'D') a
LEFT JOIN
    b_source.ord_dtl b ON TRIM(a.referenceto) = TRIM(b.referenceto)
                       AND TRIM(a.refno) = TRIM(b.refno)
                       AND b.delete_flag <> 'D'
"""
```

Step 2: Generating SQL for Main Sales Orders Table
The `generate_sales_orders_sql` function generates a SQL query for creating the `sales_orders_comp` main table. This table is created by joining multiple tables from the `s_master` and `b_source` databases.

```
sql = """
CREATE OR REPLACE TABLE sales_orders_comp AS
SELECT
    reg.StdValue AS CompSourceSystem,
    a.OrderNumber AS CompOrderNumber,
    ...
FROM
    s_master.sale_orders a
LEFT JOIN
    s_master.sale_orders b ON a.OrderNumber = b.OrderNumber AND a.Li
neNumber = b.LineNumber
LEFT JOIN
    (SELECT StdValue
     FROM s_master.ord_reg
```

```
    WHERE TRIM(Type) = 'ALL'
    AND TRIM(RegCode) = 'SOURCE_SYSTEM_CODE'
    AND TRIM(sourceSystem) = 'GLOBAL'
    AND TRIM(sourceSystemValue) = 'SAP') reg
"""
```

Step 3: Executing SQL Queries using Apache Spark
The `execute_sql` function takes a SparkSession and a SQL query as input and executes the query using the `spark.sql` method.

```
def execute_sql(spark, sql_query):
    return spark.sql(sql_query)
```

Step 4: Main Function to Generate and Execute SQL Queries
The `main` function creates a SparkSession, generates the SQL queries for the intermediate staging and main sales orders tables, and prints the generated SQL queries. It also provides an option to execute the SQL queries using the `execute_sql` function.

```
def main():
    spark = SparkSession.builder
        .appName("Sales Orders SQL Generator")
        .enableHiveSupport()
        .getOrCreate()

    stg_sql = generate_sales_orders_stg_sql()
    print("=== SQL for sales_orders_comp_stg ===")
    print(stg_sql)

    main_sql = generate_sales_orders_sql()
    print("n=== SQL for sales_orders_comp ===")
    print(main_sql)

    # execute_sql(spark, stg_sql)
    # execute_sql(spark, main_sql)
```